



BloodbankReport

Site: <http://localhost:3001>

Generated on Sun, 16 Nov 2025 20:02:32

ZAP Version: 2.16.1

ZAP by [Checkmarx](#)

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	2
Low	2
Informational	1

Alerts

Name	Risk Level	Number of Instances
Content Security Policy (CSP) Header Not Set	Medium	1
Missing Anti-clickjacking Header	Medium	1
Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s)	Low	3
X-Content-Type-Options Header Missing	Low	3
Modern Web Application	Informational	1

Alert Detail

Medium	Content Security Policy (CSP) Header Not Set
Description	Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page — covered types are JavaScript, CSS, HTML frames, fonts, images and embeddable objects such as Java applets, ActiveX, audio and video files.
URL	http://localhost:3001/display.html
Method	GET
Attack	
Evidence	
Other Info	
Instances	1

Solution	Ensure that your web server, application server, load balancer, etc. is configured to set the Content-Security-Policy header.
Reference	https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html https://www.w3.org/TR/CSP/ https://w3c.github.io/webappsec-csp/ https://web.dev/articles/csp https://caniuse.com/#feat=contentsecuritypolicy https://content-security-policy.com/
CWE Id	693
WASC Id	15
Plugin Id	10038

Medium	Missing Anti-clickjacking Header
Description	The response does not protect against 'ClickJacking' attacks. It should include either Content-Security-Policy with 'frame-ancestors' directive or X-Frame-Options.
URL	http://localhost:3001/display.html
Method	GET
Attack	
Evidence	
Other Info	
Instances	1
Solution	Modern Web browsers support the Content-Security-Policy and X-Frame-Options HTTP headers. Ensure one of them is set on all web pages returned by your site/app. If you expect the page to be framed only by pages on your server (e.g. it's part of a FRAMESET) then you'll want to use SAMEORIGIN, otherwise if you never expect the page to be framed, you should use DENY. Alternatively consider implementing Content Security Policy's "frame-ancestors" directive.
Reference	https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Frame-Options
CWE Id	1021
WASC Id	15
Plugin Id	10020

Low	Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s)
Description	The web/application server is leaking information via one or more "X-Powered-By" HTTP response headers. Access to such information may facilitate attackers identifying other frameworks/components your web application is reliant upon and the vulnerabilities such components may be subject to.
URL	http://localhost:3001/alerts.csv
Method	GET
Attack	
Evidence	X-Powered-By: Express
Other Info	
URL	http://localhost:3001/data
Method	GET
Attack	
Evidence	X-Powered-By: Express

Other Info	
URL	http://localhost:3001/display.html
Method	GET
Attack	
Evidence	X-Powered-By: Express
Other Info	
Instances	3
Solution	Ensure that your web server, application server, load balancer, etc. is configured to suppress "X-Powered-By" headers.
Reference	https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/01-Information_Gathering/08-Fingerprint_Web_Application_Framework https://www.troyhunt.com/shhh-dont-let-your-response-headers/
CWE Id	497
WASC Id	13
Plugin Id	10037

Low	X-Content-Type-Options Header Missing
Description	The Anti-MIME-Sniffing header X-Content-Type-Options was not set to 'nosniff'. This allows older versions of Internet Explorer and Chrome to perform MIME-sniffing on the response body, potentially causing the response body to be interpreted and displayed as a content type other than the declared content type. Current (early 2014) and legacy versions of Firefox will use the declared content type (if one is set), rather than performing MIME-sniffing.
URL	http://localhost:3001/alerts.csv
Method	GET
Attack	
Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
URL	http://localhost:3001/data
Method	GET
Attack	
Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
URL	http://localhost:3001/display.html
Method	GET
Attack	
Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.

Instances	3
Solution	Ensure that the application/web server sets the Content-Type header appropriately, and that it sets the X-Content-Type-Options header to 'nosniff' for all web pages. If possible, ensure that the end user uses a standards-compliant and modern web browser that does not perform MIME-sniffing at all, or that can be directed by the web application /web server to not perform MIME-sniffing.
Reference	https://learn.microsoft.com/en-us/previous-versions/windows/internet-explorer/ie-developer/compatibility/gg622941(v=vs.85) https://owasp.org/www-community/Security_Headers
CWE Id	693
WASC Id	15
Plugin Id	10021

Informational	Modern Web Application
Description	The application appears to be a modern web application. If you need to explore it automatically then the Ajax Spider may well be more effective than the standard one.
URL	http://localhost:3001/display.html
Method	GET
Attack	
	<pre> <script> const PAGE_HOLD_MS = 4200; const SLIDE_MS = 800; const REFRESH_MS = 10000; const ALERT_DISPLAY_MS = 3000; const tbody = document.getElementById("bloodBody"); const viewport = document.getElementById("scrollViewport"); const empty = document.getElementById("empty"); const meta = document.getElementById("meta"); const pageInfo = document.getElementById("pageInfo"); const toastContainer = document.getElementById("toast-container"); let bloodData = []; let currentIndex = 0; let rowHeight = 0; let pageTimer = null; let refreshTimer = null; let lastHash = ""; let rowsPerPage = 0; let alertMessages = []; let currentAlertIndex = 0; let alertTimer = null; let isAlertAnimating = false; function sleep(ms, cb) { setTimeout(cb, ms); } function cycleAlerts() { if (alertMessages.length === 0 isAlertAnimating) { return; } isAlertAnimating = true; const currentMessage = toastContainer.querySelector('.toast-message'); if (currentMessage) { currentMessage.classList.add('exiting'); currentMessage.classList.remove('active'); setTimeout(() => { currentMessage.remove(); }, 500); } const newMessage = document.createElement('div'); newMessage.className = 'toast-message'; newMessage.innerHTML = alertMessages[currentAlertIndex]; toastContainer.appendChild(newMessage); setTimeout(() => { newMessage.classList.add('active'); }, 100); currentAlertIndex = (currentAlertIndex + 1) % alertMessages.length; setTimeout(() => { isAlertAnimating = false; }, 1000); } async function fetchAlerts() { try { const response = await fetch('/alerts.csv'); if (!response.ok) { throw new Error('HTTP error, status = \${response.status}'); } const csvText = await response.text(); const lines = csvText.split('\n').slice(1); alertMessages = lines.map(line => line.trim()).filter(line => line).map(line => line.slice(1, -1)); if (alertMessages.length > 0) { cycleAlerts(); if (alertMessages.length > 1) { alertTimer = setInterval(cycleAlerts, ALERT_DISPLAY_MS); } } } catch (error) { console.error("Error fetching alerts:", error); } } async function fetchData() { try { const response = await fetch('/data'); if (!response.ok) { throw new Error('HTTP error, status = \${response.status}'); } const data = await response.json(); return data; } catch (error) { console.error("Error fetching data:", error); return null; } } function hashData(list) { return JSON.stringify(list.map(item => `\${item.Name}-\${item.kn_name}-\${item.Status}`)); } function setScrollTop(yPx, withAnimation) { if (withAnimation) { const start = viewport.scrollTop; const end = yPx; const duration = SLIDE_MS; let startTime = null; function animationStep(timestamp) { if (!startTime) startTime = timestamp; const progress = timestamp - startTime; const percentage = Math.min(progress / duration, 1); viewport.scrollTop = start + (end - start) * percentage; if (progress < duration) { requestAnimationFrame(animationStep); } } requestAnimationFrame(animationStep); } else { viewport.scrollTop = yPx; } } function renderRows(data) { return new Promise(resolve => { tbody.innerHTML = ""; if (!data data.length === 0) { empty.style.display = "block"; resolve(); return; } empty.style.display = "none"; for (let i = 0; i < data.length; i++) { const item = data[i]; const tr = document.createElement("tr"); const tdName = document.createElement("td"); const englishName = document.createElement("div"); englishName.textContent = (item.Name "-").toUpperCase(); tdName.appendChild(englishName); if (item.kn_name) { const kannadaName = document.createElement("div"); kannadaName.className = 'kannada-text'; kannadaName.textContent = item.kn_name; tdName.appendChild(kannadaName); } tr </pre>

Evidence	<pre> appendChild(tdName); const tdAge = document.createElement("td"); tdAge.textContent = item.Age "-"; tr.appendChild(tdAge); const tdComponent = document.createElement("td"); const componentGrid = document.createElement("div"); componentGrid.className = "component-grid"; const labels = [{ en: "PRBC", kn: " ", key: "PRBC" }, { en: "FFP", kn: " ", key: "FFP" }, { en: "PLATELET CONCENTRATE", kn: " ", key: "PLATELET_CONCENTRATE" }, { en: "CRYOPRECIPITATE", kn: "", key: "CRYOPRECIPITATE" }, { en: "CRYOPOOR PLASMA", kn: " ", key: "CRYOPOOR_PLASMA" }]; const neededComponents = labels.filter(label => item[label. key]); const count = neededComponents.length; if (count === 1) { componentGrid.style. gridTemplateColumns = "1fr"; } else if (count === 2) { componentGrid.style. gridTemplateColumns = "1fr 1fr"; } else if (count >= 3) { componentGrid.style. gridTemplateColumns = "1fr 1fr"; } neededComponents.forEach(row => { const gridItem = document.createElement("div"); gridItem.className = "component-grid-item"; const englishDiv = document.createElement("div"); englishDiv.textContent = `\${row.en} - \${item [row.key]}`; gridItem.appendChild(englishDiv); if (item[`\${kn}_\${row.key}`]) { const kannadaDiv = document.createElement("div"); kannadaDiv.className = "kannada-text"; kannadaDiv. textContent = `\${row.kn} - \${item[`\${kn}_\${row.key}`]}`; gridItem.appendChild(kannadaDiv); } componentGrid.appendChild(gridItem); }); tdComponent.appendChild(componentGrid); tr. appendChild(tdComponent); const tdStatus = document.createElement("td"); tdStatus. textContent = item.Status "-"; if (item.Status && item.Status.toLowerCase() === 'emergency') { tdStatus.classList.add('status-emergency'); } tr.appendChild(tdStatus); tbody. appendChild(tr); } requestAnimationFrame(() => { const firstRow = tbody.querySelector("tr"); if (firstRow) { const style = window.getComputedStyle(firstRow); rowHeight = parseFloat (style.height); } if (rowHeight > 0) { rowsPerPage = Math.floor(viewport.clientHeight / rowHeight); } currentIndex = 0; setScrollTop(0, false); resolve(); }); } function goToPage (pageIdx) { if (rowsPerPage === 0) return; currentIndex = Math.min(pageIdx * rowsPerPage, Math.max(0, bloodData.length - 1)); const offset = currentIndex * rowHeight; setScrollTop(offset, true); const pageCount = Math.ceil(bloodData.length / rowsPerPage); pageInfo.textContent = `\${Math.floor(currentIndex / rowsPerPage) + 1} of \${pageCount} • \${bloodData.length} items`; } function stepPager() { if (document.hidden bloodData.length <= rowsPerPage) return; const pageCount = Math.ceil(bloodData.length / rowsPerPage); const currentPage = Math.floor(currentIndex / rowsPerPage); let nextPage = (currentPage + 1); if (nextPage >= pageCount) { sleep(SLIDE_MS, () => setScrollTop(0, false)); nextPage = 0; } goToPage(nextPage); } function startPager() { stopPager(); pageTimer = setInterval (stepPager, PAGE_HOLD_MS); } function stopPager() { if (pageTimer) clearInterval (pageTimer); pageTimer = null; } async function refreshLoop() { const payload = await fetchData(); if (!payload) { meta.textContent = "Error fetching data"; return; } const list = Array.isArray(payload) ? payload : []; const newHash = hashData(list); if (newHash !== lastHash) { bloodData = list; await renderRows(bloodData); lastHash = newHash; goToPage (0); } meta.textContent = `Last updated: \${new Date().toLocaleTimeString('en-US', { hour: 'numeric', minute: 'numeric', hour12: true })}`; } function startRefresh() { stopRefresh(); refreshLoop(); refreshTimer = setInterval(refreshLoop, REFRESH_MS); } function stopRefresh() { if (refreshTimer) clearInterval(refreshTimer); refreshTimer = null; } document. addEventListener("visibilitychange", () => { if (document.hidden) { stopPager(); } else { renderRows(bloodData).then(() => { startPager(); }); }); window.addEventListener ("resize", () => { if (!bloodData.length) return; renderRows(bloodData).then(() => { goToPage (0); }); }); fetchAlerts(); startRefresh(); startPager(); </script> </pre>
Other Info	No links have been found while there are scripts, which is an indication that this is a modern web application.
Instances	1
Solution	This is an informational alert and so no changes are required.
Reference	
CWE Id	
WASC Id	
Plugin Id	10109